Herald

Herald — ATMOSphere Workable-Items Integration (Operator Guide)

Field	Value
Revision	2
Created	2026-05-29
Last modified	2026-05-31
Status	active
Status summary	r2: documented the participant/attribution contract (`docs/design/PARTICIPANT_ATTRIBUTION.md`) — new §3.6 (`created_by`/`assigned_to` columns), §3.7 (the HERALD_<CHANNEL>_OPERATOR_USERNAME operator env var + per-messenger `subscribers/subscriber_aliases.username` mapping), §3.8 (the @-tagging behaviour matrix). Operator-facing reference for the ATMOSphere⍰Herald workable-items integration (Phase 2 build). Documents the OUTBOUND flow (workable-items SQLite SSoT + Markdown trackers → `commons_watch` → `commons_workable.Diff` → `pherald/internal/workflow` bridge → existing `pherald/internal/runner` fan-out → channels) and the INBOUND flow (operator message → Claude Code dispatch → <<<HERALD-REPLY>>> action router → `ItemMutator` / investigation-with-confirmation). Grounded in the master plan `~/Documents/ATMOSphere_Herald_Integration_Plan.md`. ANTI-BLUFF: built-and-tested pieces are marked LIVE; not-yet-built or externally-gated pieces are marked PLANNED / PENDING and never implied to work.
Issues	HRD-150, HRD-151 (formatter polish), HRD-154, HRD-155, HRD-156, HRD-157, HRD-158, HRD-131
Issues summary	DB materialization + MD⍰SQLite regenerator + preference routing + full-automation test suite + ATMOSphere registration + covenant propagation are still open — see §1.3 + §7.
Fixed	(n/a — new guide)
Continuation	bump when `docs/workable_items.db` is materialized (HRD-155/HRD-131 Phase 3), when the MD⍰SQLite regenerator lands (HRD-150), when preference/quiet-hours routing is enforced (HRD-154), when the live MTProto round-trip evidence lands (HRD-156), and when Herald is registered as a `tools/herald` submodule in ATMOSphere (HRD-157).

Table of contents

---

§1. Overview

§1.1 What the integration does

The ATMOSphere?Herald integration connects ATMOSphere's workable-items tracking system (the project's issue/ticket lifecycle) to its operators via Herald, in two directions:

- **Outbound (watch → notify).** Herald watches the workable-items SQLite single-source-of-truth (SSoT) plus its `Issues.md` / `Fixed.md` Markdown trackers. On every change it computes a per-property diff and fans a notification out to Subscribers over channels (Telegram primary) — Jira/ClickUp-style: item creation, each property change (with the exact old→new value), status transitions, content updates, and deletions.
- **Inbound (message → act).** Operators message the channel and Herald routes the message through a Claude Code dispatch into a structured `<<<HERALD-REPLY>>>` action. Workable-item CRUD (`item.update` / `item.delete`) and investigations (`investigation.start`) are supported. Mutating actions proposed by an investigation run only after an explicit `CONFIRM <token>` reply (act-with-confirmation).

Because both directions converge on the same `commons_workable` store, an operator-driven update produces the same per-property diff notification a file edit would — Subscribers always see "what changed exactly" regardless of who changed it.

§1.2 The requirements (R1–R7) and their honest status

The master plan (`~/Documents/ATMOSphere_Herald_Integration_Plan.md` §2.3) defines seven requirements. Current Herald-side status:

Req	Capability	Status	Where
R1	Watch the SQLite SSoT (create / update / delete)	**BUILT (LIVE)**	`commons_workable` (store + diff change-feed) + `commons_watch` (fsnotify + WAL-poll). Tested green.
R2	Watch MD trackers + keep in sync with the SSoT	**PARTIAL**	The watcher watches the MD trackers (`commons_watch`), and the parser reads them (`commons_workable/parser.go`). The bidirectional MD?SQLite regenerator + drift resolution is **PLANNED** (HRD-150).
R3		**BUILT (LIVE)**	

Req	Capability	Status	Where
	Emit a notification per event with the exact per-property diff		`commons_workable.Diff` (per-property change-feed) + `pherald/internal/workflow` (CloudEvent mapper + diff renderer). Tested green.
R4	Subscribers receive over channels (Telegram primary)	**BUILT (transport reused)**	`pherald/internal/runner` fan-out, driven by `workflow.Notifier`. NOTE: full PG-backed subscriber resolution is **PENDING** (HRD-156); `pherald watch` currently fans out to the configured channel targets directly (see §4.4).
R5	Inbound → process → act (update / extend / create / investigate / return)	**PARTIAL (LIVE for the implemented actions)**	`item.update` / `item.delete` / `investigation.start` are wired in `pherald/internal/inbound` with act-with-confirmation. Investigation autonomy scope is an open operator decision (plan §8.6).
R6	Coherent full CRUD driven by BOTH SSoT-change (notify) and inbound (mutate)	**PARTIAL**	The inbound write path (`ItemMutator` → `commons_workable.Repo`) and the outbound watch path both use `commons_workable`. The closing seam — having an inbound mutation also regenerate the MD trackers and re-emit through the watcher in one process — depends on the regenerator (HRD-150) and the daemon co-residency wiring.
R7	Full-automation anti-bluff tests with physical evidence	**PARTIAL**	Hermetic unit + real-SQLite + real-fsnotify + real-dispatcher tests are green (§7). The live MTProto Telegram round-trip + stress/chaos + paired §1.1 mutation gate suite is **PENDING** (HRD-156).

§1.3 What is NOT yet done (do not assume it works)

PLANNED / PENDING, explicitly:

- **The SQLite SSoT file does not exist yet.** No `docs/workable_items.db` is materialized in this repo. Materialization is **PENDING** (HRD-155 + HRD-131 Phase 3). See §3.5.
- **The MD⍰SQLite bidirectional regenerator** (R2 / HRD-150) is **PLANNED**. Today the parser reads the trackers and the watcher watches them, but Herald does not yet write the trackers back from the DB or resolve drift between them.
- **PG-backed subscriber resolution** for the watch path is **PENDING** (HRD-156). `pherald watch` fans out to the configured channel targets, not yet to a PG `Subscriber` set.
- **Preference / quiet-hours routing** (HRD-154) is **PLANNED** — the `PreferenceSet` / `QuietHours` types exist but the resolver does not yet honour them.
- **The full-automation anti-bluff test suite** with live Telegram evidence via MTProto (HRD-156) is **PENDING** — it requires the operator credential bootstrap. Until then the live tests honestly SKIP-with-reason (§11.4.3), they do not PASS.
- **ATMOSphere-side registration** (HRD-157): Herald is not yet a proper submodule of ATMOSphere (the existing `./herald` gitlink is a broken orphan stub). Phase 3 replaces it with a `tools/herald` submodule. This is **PLANNED**.
- **Covenant verbatim-phrase propagation** into ATMOSphere `QWEN.md` files (HRD-158) is **PLANNED**.

§2. Architecture

§2.1 Outbound — SSoT change → Subscriber notification (LIVE)

```
ATMOSphere workable-items                  Herald (commons_workable + common
─────────────────────────                  ────────────────────────────────
docs/Issues.md  ┐                           commons_watch.Watcher
docs/Fixed.md   ├─ edits ─▶ workable_       │  fsnotify on .db + .db-wal + .
                ┘  items.db │  (+ sync)      │   PLUS a WAL-poll fallback (mt
                  (SQLite SSoT)              │   the -wal sidecar so the main
                                             ▼
                                 pherald watch — runWatch loop
                                   snapshot prev := Repo.List(Issues) + Repo.
                                   on every watch Event (or safety-net reconc
                                     curr := re-list  →  commons_workable.Dif
                                     prev := curr
                                             │  []workable.Change  (per-prop
                                             ▼
                                 pherald/internal/workflow
                                   ChangesToEvents  → 1 CloudEvent per Change
                                                      (type digital.vasic.her
                                   RenderChange     → 1 Jira/ClickUp-style di
                                   Notifier.Notify  → feeds each rendered lin
                                             ▼
                                 pherald/internal/runner.ChannelDispatcher  (
                                   per-recipient commons.Channel.Send  +  del
                                             ▼
                                 Telegram main group / per-subscriber channel
                                   "🆕 ATM-238 created"
                                   "🔄 ATM-238 status: In progress → Ready for
                                   "✏️ ATM-238 severity: Critical → Medium"
```

Design facts (verified in code):

- **The DB is the change-detection anchor.** Herald keeps its own prior-state snapshot keyed on the composite `(atm_id, current_location)` and diffs consecutive snapshots — it does not rely on the tool's `item_history` (which is not field-level and is unpopulated).
- **WAL handling is explicit.** `pherald watch` adds the `-wal` / `-shm` sidecars of every `.db` path to the watch set (`withWALSidecars`) and runs a safety-net reconcile ticker (default 1s) so a logical mutation is detected even when the main inode does not emit a timely fsnotify event.
- **The fan-out is reused verbatim.** `workflow.Notifier` owns no delivery logic; it drives the production `runner.ChannelDispatcher.Process` → `commons.Channel.Send` path. Only the change→CloudEvent producer and the diff renderer are new.

§2.2 Inbound — operator message → action → CRUD / investigation → reply (LIVE for implemented actions)

```
Operator → channel: "ATM-238 set status Ready for testing; add note: retri
           │
           ▼
```

```
pherald listen  (REUSED inbound runtime)
  classify → Claude Code dispatch (Opus-pinned, verbatim envelope pre-text
           → ParseReply: extract <<<HERALD-REPLY>>> JSON → typed Reply{Act
        │ Reply.Action ∈ { reply, issue.open, event.emit,           ← pre
        │                  item.update, item.delete, investigation.start }
        ▼
  Dispatcher action registry (handlers map)
    item.update          → ItemMutator.Update(atm_id, location, fields) →
    item.delete          → ItemMutator.Delete(atm_id, location)         →
    investigation.start  → build report; if it proposes a mutation, record
                           and reply "Reply CONFIRM <token> to apply: …"
    CONFIRM <token>      → pendingStore.take(token) → ItemMutator.Update/D
```

Design facts (verified in code):

- **The 3-way action switch became an extensible registry.** `pherald/internal/inbound/dispatcher.go` routes by `Reply.Action` through a `handlers` map; the workable actions are registered alongside the original three.
- `ItemMutator` **is the single inbound write surface** (`item_mutator.go`), parallel to the existing `IssueOpener`. Production binds `RepoMutator` over a real `commons_workable.Repo`; unit tests bind a recording fake. `RepoMutator.Update` reads the row, applies the field deltas, and writes it back through `Repo.Update` — a missing row surfaces as `workable.ErrNotFound`, an invalid status is rejected by the closed-set check, and an unknown column name is rejected loudly (no silent no-op).
- `investigation.start` **is act-with-confirmation** (operator decision 2026-05-29). A proposed mutation is recorded in a `pendingStore` under a token; the mutation runs only on a subsequent `CONFIRM <token>` message. The entry is consumed on lookup so a replayed `CONFIRM` cannot double-apply. A report-only investigation (no proposed mutation) emits no prompt and stores nothing.

§2.3 Module / layer placement

Unit	Layer	Status	Responsibility
`commons_workable/`	L1 foundation	**BUILT**	SQLite open + canonical schema, full CRUD repo, per-property diff change-feed, ATMOSphere Markdown-tracker parser
`commons_watch/`	L1 foundation	**BUILT**	fsnotify wrapper + debounce-coalesce + SQLite-WAL poll fallback
`pherald/internal/workflow/`	flavor-internal	**BUILT**	change→CloudEvent mapper, Jira/ClickUp diff renderer, `Notifier` over the real `ChannelDispatcher`
`pherald/internal/inbound/` (workable extension)	flavor-internal	**BUILT**	action registry + `item.update`/`item.delete`/`investigation.start` + `ItemMutator` + pending/confirm flow

Unit	Layer	Status	Responsibility
`pherald watch`	flavor binary subcommand	**BUILT**	the watch → diff → notify daemon entrypoint
MD?SQLite regenerator	`commons_workable` + scripts	**PLANNED (HRD-150)**	bidirectional sync; references `constitution/scripts/workable-items/` per §11.4.74

---

§3. The SQLite single-source-of-truth

§3.1 Canonical path

The canonical DB path is `docs/workable_items.db` (no leading dot). `pherald watch` resolves it from, in order: the `--db` flag, the `HERALD_WORKABLE_DB` env var, then the `docs/workable_items.db` default.

> Note: HRD-131 historically referenced `docs/.workable_items.db` (leading dot). The constitution §11.4.95 canonical path is `docs/workable_items.db` and the code uses the dot-less form; reconcile to the dot-less canonical path when the DB is materialized (HRD-131 Phase 2+).

§3.2 Schema (mirrored verbatim from ATMOSphere)

`commons_workable.Open(path)` opens (creating if absent) the DB, sets `PRAGMA journal_mode=WAL` + `PRAGMA foreign_keys=ON`, pins a single connection (so connection-scoped PRAGMAs hold), and applies the schema idempotently (`CREATE TABLE IF NOT EXISTS`). The driver is the pure-Go `modernc.org/sqlite` (no CGO).

Three tables:

- `items` — composite primary key `(atm_id, current_location)`. Columns: `atm_id`, `type` (`CHECK IN ('Bug','Feature','Task')`), `status`, `severity`, `title`, `description`, `forensic_anchor`, `closure_criteria`, `composes_with` (JSON array as TEXT), `current_location` (`CHECK IN ('Issues','Fixed')` default `'Issues'`), `body_md`, `created_at`, `last_modified`.
- `item_history` — append-only audit (`event_type IN ('Opened','Updated','Reopened','Fixed','Implemented','Completed','Obso` `by IN ('AI','User')`, `on_date`, `reason`, `evidence_path`, `created_at`). NOTE: this table is schema-defined but Herald does NOT rely on it for diffs — it is not field-level and is currently unpopulated. Herald computes diffs from its own prior-state snapshot.
- `meta` — key/value with `last_modified`.

§3.3 The 10-value status closed set

`commons_workable.StatusValues` is the canonical closed set; `Create`/`Update` reject any status outside it (no silent acceptance):

```
Queued
In progress
Ready for testing
```

```
In testing
Reopened
Operator-blocked
Fixed (→ Fixed.md)
Implemented (→ Fixed.md)
Completed (→ Fixed.md)
Obsolete (→ Fixed.md)
```

Types are the closed set `Bug | Feature | Task` (enforced by the schema CHECK).

§3.4 The parser and the format match

`commons_workable.ParseTracker(markdown, location)` reads ATMOSphere's real tracker format directly — it does NOT require the `## ABC-123 — title` shape the constitution tool's parser expects. It accepts H2 headings in shapes like:

```
## §GL CRITICAL — [ATM-238] Netflix login failure on D3
## SYS — [ATM-101] Disk pressure alerting
## §UX — Tidy the onboarding copy
## A. Global blockers            (section header — skipped)
```

Rules: an item is an H2 block whose body contains a `**Status:**` line; a heading with no `**Status:**` is treated as a section header and skipped. The `[ATM-NNN]` bracket is the id; a bracket-less item heading gets a stable derived id (`ATM-DERIVED-<8hex>` from a sha1 of the heading). `**Status:**` / `**Type:**` / `**Severity:**` metadata lines populate those fields; the raw body block becomes `body_md`.

§3.5 Relationship to the trackers and the constitution tool — PENDING materialization

- The Markdown trackers (`docs/Issues.md` / `docs/Fixed.md`) are today the live source; the **DB does not exist yet** (no `docs/workable_items.db`).
- Materialization is **PENDING (HRD-155 + HRD-131 Phase 3)**: build/operate the constitution workable-items tool (`constitution/scripts/workable-items/`), supply the ATMOSphere-format parser the tool lacks, run `sync md-to-db` against the real trackers, `validate`, and commit the DB per §11.4.95 (version-controlled SSoT; only the `-wal`/`-shm` sidecars gitignored).
- Per §11.4.74 (catalogue-first), Herald references the constitution tool for the implemented `sync` / `diff` / `validate` rather than reimplementing them; the regenerator + drift resolution is HRD-150.

§3.6 Participant attribution — `created_by` / `assigned_to` (per `docs/design/PARTICIPANT_ATTRIBUTION.md`)

Per the participant/attribution contract (docs/design/PARTICIPANT_ATTRIBUTION.md, inherited from HelixConstitution per §11.4.35), the `items` table gains two attribution columns:

Column	Type	Holds
`created_by`	`TEXT NOT NULL DEFAULT ''`	the **canonical handle** of whoever opened/assigned the item
`assigned_to`	`TEXT NOT NULL DEFAULT ''`	the **canonical handle** of the responsible participant (default = the operator)

A **canonical handle** is a messenger-neutral string: either the reserved sentinel Claude (the system agent — kind=agent, NEVER tagged) or a human's canonical handle (which defaults to their Telegram @username since Telegram is the primary messenger). The handle is the same value that appears in the Markdown trackers, so the SSoT round-trips byte-identically.

Who sets created_by:

- Opened via the **Claude Code CLI prompt** (operator-driven) → created_by = OperatorHandle() (see §3.7).
- Opened by **System/Claude** detecting an issue/task/improvement/missing-feature → created_by = "Claude".
- Received **through Herald** (an inbound subscriber message that opens/updates the item) → created_by = the sender's resolved canonical handle (IdentityResolver.ResolveSender maps the message's per-channel @username + ids to the canonical handle).

assigned_to defaults to OperatorHandle() and may be overridden explicitly (a prompt or inbound message that assigns to @someoneelse).

In the Markdown trackers these surface as **Created-By:** <handle> / **Assigned-To:** <handle> (ATMOSphere heading-format) or as **Created-By** / **Assigned-To** pipe-table columns (Herald Issues.md/Fixed.md); the parser reads them and validate accepts empty values for legacy items. commons_workable.Item carries CreatedBy + AssignedTo, and the change-feed Diff emits an item.field.changed for either column.

§3.7 Operator env var + per-messenger username mapping

The **operator** is the one human who drives the system via the Claude Code CLI. They are designated by an env var, NOT a DB flag:

Env var	Example value	Meaning
HERALD_TGRAM_OPERATOR_USERNAME	@milos85vasic	the operat Telegram @userna (primary messenger
HERALD_<CHANNEL>_OPERATOR_USERNAME	HERALD_SLACK_OPERATOR_USERNAME=…	per-messenger generaliza for any oth channel

The operator's canonical handle equals this env value; IdentityResolver.OperatorHandle() returns it. The operator is otherwise a normal Participant.

A **Participant** (logical Subscriber/User) may have a DIFFERENT username on every messenger. The mapping is held in PG: subscribers carries the logical party (canonical handle, display_name, kind ∈ {human, agent, service}), and subscriber_aliases carries the per-channel handle (subscriber_id, channel, channel_user_id, + **NEW** username TEXT — the per-channel @handle used for tagging, distinct from channel_user_id, which is the chat/user id; UNIQUE (channel, channel_user_id)). Resolution: ResolveSender(channel, channel_user_id, `username`) maps an inbound message to a canonical handle; `UsernameFor(handle, channel)` returns the `@username` for a canonical handle on a target channel (and reports not-found when the participant has no alias on that channel — you cannot tag someone who is not on that messenger).

§3.8 Tagging behaviour on notifications

On any workable-item event, the outbound notification to each channel @-tags the participant(s) who must be aware, resolved to that channel's `@username`:

- tag `assigned_to` when it is a human handle AND `assigned_to != Operator`;
- tag `created_by` when it is a human handle AND `created_by != Operator` AND `created_by != "Claude"`;
- `"Claude"` is NEVER tagged (it is the system); the Operator is NEVER tagged (no self-ping);
- de-dup the set, then resolve each handle to the channel's `@username` and skip any participant with no alias on that channel.

So: assigned-to-Operator → no tag; opened-by-Operator-assigned-to-another → tag the assignee; opened-by-a-non-Operator-non-Claude subscriber → tag them. The `tgram` adapter renders a mention as `@username`; other adapters render their channel's native mention syntax (future).

---

§4. Running `pherald watch`

> Prerequisite: a materialized `docs/workable_items.db`. Until HRD-155/HRD-131 Phase 3 land, the DB does not exist, so a real run depends on that PENDING work. `pherald watch` will create an empty schema-only DB if the path is absent, but it will have no items to diff.

§4.1 Command

```
pherald watch [flags]
```

Long-running. It (1) opens the SSoT and snapshots every item at the watched locations (`Issues` + `Fixed`), (2) starts a `commons_watch.Watcher` over the DB file (+ WAL sidecars) and the trackers, (3) on every change re-lists, diffs against the prior snapshot, renders each per-property delta, and fans it out through the production `ChannelDispatcher`. SIGINT/SIGTERM cancels the loop cleanly.

§4.2 Flags

Flag	Default	Meaning
`--db <path>`	`$HERALD_WORKABLE_DB`, else `docs/workable_items.db`	Workable-items SQLite DB path
`--issues <path>`	`docs/Issues.md`	Issues.md tracker path (watched)
`--fixed <path>`	`docs/Fixed.md`	Fixed.md tracker path (watched)
`--poll <duration>`	`1s`	

Flag	Default	Meaning
		WAL-poll fallback + safety-net reconcile interval (`0` disables polling, fsnotify only)

§4.3 Environment

Variable	Used by	Meaning
`HERALD_WORKABLE_DB`	`pherald watch`	DB path fallback when `--db` is unset
`HERALD_CHANNELS`	channel setup (shared with `pherald listen`)	Comma-separated enabled channels (e.g. `tgram`)
per-channel namespaced env	channel setup	Credentials + target per channel (e.g. the Telegram bot token + target chat id). See `docs/guides/MESSENGER_CHANNELS.md` §2–§4 and `docs/guides/OPERATOR_CREDENTIALS.md`.
`HERALD_PROJECT_NAME`	dispatch envelope	The Herald project name; for the ATMOSphere deployment, `ATMOSphere`.

The MTProto user-account credentials used by the live test harness (`qaherald`) are configured in the ATMOSphere `.env` per `docs/guides/OPERATOR_CREDENTIALS.md`; they are a test-driver bootstrap, not a `pherald watch` runtime dependency.

§4.4 What subscribers receive — current fan-out caveat

`pherald watch` derives its recipient set from the per-channel configured `Target` (the chat/channel id), so it notifies the operator channel directly. Full PG-backed `Subscriber` resolution is **PENDING (HRD-156)**; until it lands, watch fans out to the configured channel targets, mirroring the explicit-recipient bypass that `workflow.NewNotifier` documents. Preference / quiet-hours filtering is **PLANNED (HRD-154)** and is not yet applied.

Subscribers receive one message per change, using the formats in §6 (🆕 created, 🔄 status, ✏️ field, 📝 content, 🗑️ removed).

---

§5. Inbound actions

Inbound runs under `pherald listen` (the existing Wave 6 runtime). The operator message is dispatched to Claude Code, whose reply must contain a <<<HERALD-REPLY>>> block followed by a JSON object. `ParseReply` extracts it into a typed `Reply`; `Action` defaults to `"reply"` when omitted. A missing marker or malformed JSON is an explicit error — never a fabricated reply (§107 anti-bluff).

§5.1 The <<<HERALD-REPLY>>> schema (workable actions)

`item.update` — apply column→value deltas to one item:

```
<<<HERALD-REPLY>>>
{
```

```
  "action": "item.update",
  "item_update": {
    "atm_id": "ATM-238",
    "location": "Issues",
    "fields": { "status": "Ready for testing", "severity": "Medium" }
  }
}
```

Updatable fields: `type`, `status`, `severity`, `title`, `description`, `forensic_anchor`, `closure_criteria`, `composes_with`, `body_md`, `last_modified`. The composite-key columns (`atm_id`, `current_location`) are NOT updatable in place — a move between locations is a delete + create. An unknown field name is rejected loudly.

`item.delete` — remove one item by composite key:

```
<<<HERALD-REPLY>>>
{
  "action": "item.delete",
  "item_delete": { "atm_id": "ATM-238", "location": "Issues" }
}
```

`investigation.start` — gather info, return a report, optionally propose ONE machine-executable mutation (deferred behind confirmation):

```
<<<HERALD-REPLY>>>
{
  "action": "investigation.start",
  "investigation": {
    "topic": "Why is ATM-238 still failing on D3?",
    "proposed_actions": ["Re-run the Netflix login flow", "Capture logcat"
    "proposed_action": {
      "kind": "update",
      "atm_id": "ATM-238",
      "location": "Issues",
      "fields": { "status": "Reopened" }
    }
  }
}
```

When `proposed_action` is omitted, the investigation is report-only — no confirmation prompt, no pending action.

§5.2 The act-with-confirmation flow

1. `investigation.start` with a `proposed_action` records the proposal in the `pendingStore` under a token and replies with a report ending in: Reply CONFIRM <token> to apply: <kind> <atm_id>/<location>
2. The operator replies CONFIRM <token> (the `CONFIRM` keyword is case-insensitive; the token is the next whitespace-delimited field, taken verbatim).
3. The dispatcher takes (and consumes) the pending proposal and executes it via the `ItemMutator` (`Update` or `Delete`), then replies Applied: <kind> <atm_id>/<location>.

Safety properties (verified): an unknown / already-consumed token is an explicit error (no fabricated success); consuming on take means a replayed CONFIRM cannot double-apply; if no ItemMutator is configured the path returns an explicit error rather than silently succeeding.

---

§6. Notification message formats

workflow.RenderChange produces one deterministic single-line message per Change. The renderer never panics or returns empty — an unknown Kind falls back to "<atm_id> <kind>".

Kind	Constant	Example rendered line
Item created	item.created	🆕 ATM-238 created
Status changed	item.status.changed	🔄 ATM-238 status: In progress → Ready for testing
Field changed	item.field.changed	✏️ ATM-238 severity: Critical → Medium
Content updated	item.content.updated	📝 ATM-238 content updated
Item removed	item.deleted	🗑️ ATM-238 removed

The diff engine (commons_workable.Diff) classifies a status difference as item.status.changed; severity / title / type differences each emit one item.field.changed; body_md / description differences each emit one item.content.updated. Output is deterministically ordered (by atm_id, then current_location, then a Kind rank, then field name). Each Change also maps 1:1 to a CloudEvent via workflow.ChangesToEvents: type digital.vasic.herald.workable.<kind>, subject item:<atm_id>, JSON body {atm_id, location, field, old, new}, fresh UUIDv7 id.

> The status-summary line in this guide's header calls these "Jira/ClickUp-style" diff lines. The current renderer emits the single-line forms above; richer formatting polish (multi-field grouping, attribution by/on) is tracked under HRD-151.
>
> **Participant @-tagging (per docs/design/PARTICIPANT_ATTRIBUTION.md).** The rendered notification body is prefixed/suffixed with the resolved @usernames for the participants the tagging matrix selects (see §3.8): the assigned_to and/or created_by human handles, never "Claude" and never the Operator, each resolved to the target channel's @username and skipped if the participant has no alias on that channel.

---

§7. Testing and evidence

§7.1 Hermetic coverage that exists today (LIVE, green)

All of the following pass under go test:

```
go test -count=1 ./commons_workable/... ./commons_watch/... \
                 ./pherald/internal/workflow/... ./pherald/
        internal/inbound/...
```

- `commons_workable` — real SQLite (`modernc.org/sqlite`, temp DB): `TestOpen_CreatesSchemaIdempotently`, `TestCRUD_RoundTrip`, `TestCreate_RejectsUnknownStatus`, `TestCreate_RejectsEmptyStatus`, `TestUpdate_LoudOnMissing`, `TestDelete_LoudOnMissing`; diff: `TestDiff_Created/Deleted/StatusChanged/FieldChanged_TitleSeverityType/ContentUpdated/DeterministicOrderingAcrossItems/NoChange`; parser: `TestParseTracker_RepresentativeItem/PlainPrefixItem/SectionHeaderSkipped/DerivesStableIDWhenNoBracket/ItemCount`.
- `commons_watch` — real `fsnotify` watcher + real files: `TestWatch_EmitsOnModify`, `TestWatch_DebounceCoalesces`, `TestWatch_PollFallbackDetectsSidecar`, `TestWatch_CancelNoGoroutineLeak`.
- `pherald/internal/workflow` — `TestChangesToEvents`, `TestRenderChange`, `TestNotifier_FeedsRealDispatcher` (drives the real `runner.ChannelDispatcher` through a recording `commons.Channel` sink — no mock of the bridge itself).
- `pherald/internal/inbound` — `ItemMutator` against a real SQLite store (`repo_mutator_test.go`), the action router with a recording fake, and the investigation defer / confirm-executes / replayed-confirm-rejected assertions.
- `pherald watch` — `watch_test.go` drives the real `runWatch` loop (real temp DB, real fsnotify watcher, real `Diff`, real `Notifier` over a recording channel) and asserts a real DB mutation produces a real rendered diff message through the real fan-out. The `watchDeps.Ready` channel is a startup-ordering signal only (closed after baseline + watcher start), not a pipeline mock.

These are anti-bluff in posture: the PASS bar is "a real DB mutation produces a real rendered diff message dispatched through the real fan-out", not "the process boots".

§7.2 Where evidence lands

Per §107.x, every shipping feature lands a `docs/qa/<run-id>/` transcript. The watch→notify and inbound→CRUD features land their evidence under `docs/qa/HRD-NNN-<TS>/` in the same logical work effort.

§7.3 What is PENDING

- **Live Telegram round-trip via MTProto (HRD-156).** A real SQLite mutation → real Telegram message captured by the `qaherald` MTProto user-account (`WaitForReply`), plus an inbound command → real row mutation, plus exact byte-for-byte diff-payload assertions. This requires the operator credential bootstrap (`.env` + a one-time `qaherald mtproto login`). Until then these live tests honestly SKIP-with-reason (§11.4.3) — they do not PASS.
- **Stress + chaos (§11.4.85) and the paired §1.1 mutation gate** for the integration path (`tests/test_atmosphere_integration_mutation_meta.sh`) are **PENDING (HRD-156)**. There is currently no `tests/test_*atmosphere*` / `tests/test_*workable*` shell gate in the repo.
- **HelixQA / Challenges registration** on the ATMOSphere side is **PLANNED (Phase 3 / HRD-157)**.

§8. Setup checklist and troubleshooting

§8.1 Operator pre-deploy checklist

- [ ] Constitution discoverable from this checkout (`tests/test_constitution_inheritance.sh` green).
- [ ] **PENDING:** `docs/workable_items.db` materialized + committed (HRD-155 / HRD-131 Phase 3). Until then `pherald watch` has no items to diff.
- [ ] `HERALD_CHANNELS` set and the per-channel credentials present (see `OPERATOR_CREDENTIALS.md` + `MESSENGER_CHANNELS.md`).
- [ ] `HERALD_PROJECT_NAME=ATMOSphere` for the ATMOSphere deployment.
- [ ] Hermetic tests green: `go test -count=1 ./commons_workable/... ./commons_watch/... ./pherald/internal/workflow/... ./pherald/internal/inbound/...`.
- [ ] For inbound: `pherald listen` configured with an `ItemMutator` bound to the real `commons_workable.Repo` (else `item.update`/`item.delete`/`CONFIRM` return "no ItemMutator configured").
- [ ] **PENDING:** live MTProto evidence captured + committed under `docs/qa/HRD-156-<TS>/` before claiming the live round-trip works (HRD-156).

§8.2 Troubleshooting cookbook

Symptom	Likely cause	Fix
`pherald watch: open workable DB …: ...`	DB path wrong or directory missing	Check `--db` / `HERALD_WORKABLE_DB`; ensure the parent dir exists. Note the DB itself may not be materialized yet (§3.5).
`pherald watch: no channels enabled (HERALD_CHANNELS resolved empty)`	No channels configured	Set `HERALD_CHANNELS` and the per-channel env (see `MESSENGER_CHANNELS.md`).
Mutations don't notify	WAL writes not seen	Ensure `--poll` > 0 (default 1s); the watcher watches `-wal`/`-shm` sidecars and runs a safety-net reconcile, but `--poll 0` disables both.
Notification fires but the diff is wrong/empty	Snapshot vs. parser mismatch	The diff compares `Repo.List` snapshots; confirm the rows actually changed in `items` (not just the MD tracker, until the regenerator HRD-150 lands the two can drift).
`item.update` errors `unknown/unupdatable field "X"`	Field not in the updatable set	Use a field from §5.1; composite-key columns are not updatable in place.

Symptom	Likely cause	Fix
`invalid status "X" (not in closed set)`	Status outside the 10-value set	Use a value from §3.3 verbatim (including the `(→ Fixed.md)` suffix where applicable).
`CONFIRM …: no pending action for token`	Token unknown / already consumed / wrong token	Re-run the investigation to get a fresh token; a `CONFIRM` consumes its proposal once.
`action=item.update but no ItemMutator configured`	`pherald listen` started without a mutator	Bind a `RepoMutator` over the real `commons_workable.Repo`.
Live Telegram test "fails" / skips	MTProto credentials not bootstrapped	Expected — the live suite SKIPs-with-reason until the operator bootstrap (HRD-156). It is not a PASS yet.

---

§9. References

Master plan

- `~/Documents/ATMOSphere_Herald_Integration_Plan.md` — architecture, phasing, requirements R1–R7, work-stream decomposition, open operator decisions.

Work-items (HRD) — filed in `docs/Issues.md`:

HRD	Scope	Status
HRD-148	`commons_workable` SQLite SSoT foundation	in_progress (landed + tested)
HRD-149	`commons_watch` file/DB watcher	in_progress (landed + tested)
HRD-150	bidirectional MD?SQLite regenerator + drift resolution	open (PLANNED)
HRD-151	change→CloudEvent→Runner bridge + diff message formatter	open (bridge built; formatter polish open)
HRD-152	inbound action registry + `ItemMutator` + investigation orchestrator	open (built + tested)
HRD-153	watcher daemon entrypoint (`pherald watch`)	open (built + tested)
HRD-154	preference / quiet-hours routing enforcement	open (PLANNED)
HRD-155	operationalize the workable-items SQLite tool + reconcile HRD-131	open (PENDING — external blocker for SSoT materialization)
HRD-156	anti-bluff full-automation test suite (MTProto live, stress/chaos, paired mutation, HelixQA)	open (PENDING — credential bootstrap)
HRD-157	ATMOSphere Phase-3 registration (`tools/herald` submodule), materialize DB, wire runner, live evidence	open (PLANNED)
HRD-158	anti-bluff covenant verbatim-phrase propagation to ATMOSphere `QWEN.md` files	open (PLANNED)
HRD-131	migrate text trackers to versioned SQLite SSoT (6-phase)	open (Phase 1 filed; later phases follow-on)

Source paths (Herald, verified)

- commons_workable/{store.go, item.go, crud.go, changefeed.go, parser.go} — SQLite store + schema, Item + status closed set, CRUD repo, per-property Diff, ATMOSphere tracker parser.
- commons_watch/watch.go — fsnotify + WAL-poll watcher.
- pherald/internal/workflow/workflow.go — ChangesToEvents, RenderChange, Notifier.
- pherald/internal/inbound/{reply.go, item_mutator.go, pending.go, dispatcher.go} — <<<HERALD-REPLY>>> schema, ItemMutator/RepoMutator, pending/confirm store, action routing.
- pherald/cmd/pherald/watch.go — the pherald watch daemon (runWatch loop, withWALSidecars, snapshot).

Related guides

- docs/guides/MESSENGER_CHANNELS.md — channel framework, HERALD_CHANNELS, per-channel config, inbox, self-filter.
- docs/guides/OPERATOR_CREDENTIALS.md — credential setup for every messenger + dispatcher (Telegram, Claude Code, MTProto bootstrap).

Sources verified

This guide documents Herald's own committed source code and the in-repo master plan; all claims were cross-referenced against the code paths cited in §9 on 2026-05-29. Per §11.4.99, no external-service instructions are introduced here that are not already covered (and source-verified) by MESSENGER_CHANNELS.md / OPERATOR_CREDENTIALS.md.

- Herald source tree (read 2026-05-29): commons_workable/, commons_watch/, pherald/internal/workflow/, pherald/internal/inbound/, pherald/cmd/pherald/watch.go.
- ~/Documents/ATMOSphere_Herald_Integration_Plan.md (rev 1, 2026-05-29).
- docs/Issues.md (HRD-131, HRD-148..HRD-158 rows, read 2026-05-29).

Verified 2026-05-30: internal doc — no external online sources. All claims derive from in-repo Herald source (commons_workable/, commons_watch/, pherald/internal/workflow/, pherald/internal/inbound/, pherald/cmd/pherald/watch.go) and the cited in-repo trackers; any external-service instructions are deferred to MESSENGER_CHANNELS.md / OPERATOR_CREDENTIALS.md, which carry their own §11.4.99 source-verification footers. Re-verify on Herald source breaking changes (workflow/inbound API edits, SQLite schema bumps).